

NAMAZ VAKTİ MOBİL UYGULAMA

ALGORİTMA DOKUMANI

Suleymaniye Vakfi Mizan Metodolojisi

Tüm hesaplama, bildirim ve konum algoritmalarının mobil uygulamaya aktarım rehberi

Haziran 2026

1. Giris ve Mimari Genel Bakis

Bu dokuman, mevcut web tabanlı namaz vakti uygulamasinin tum algoritmalarini ve veri modellerini icerir. Amac, bir mobil uygulama (Flutter veya React Native) gelistirilirken tum is mantiginin birebir aktarilabilmesidir. Web uygulamasinda Next.js 16 App Router uzerinde calisan bu sistem, mobil ortamda yerel hesaplama ve yerel bildirim planlama ile cok daha verimli calisacaktır.

1.1 Sistem Bilesenleri

Uygulama 5 temel modulden olusur. Her modul birbirinden bagimsiz olarak calisabilir ve mobil uygulamada ayri paketler olarak implemente edilebilir. Moduller arasi veri akisi tek yonludur: konum tespiti sonucu hesaplama motoruna gider, hesaplama sonucu bildirim sistemine ve arayuze aktarilir.

Modul	Sorumluluk	Mobil Karsiligi
Namaz Vakti Hesaplama	Astronomik hesaplama, 12 yontem destegi	Yerel Dart/Swift/Kotlin sinifi
Konum Tespiti	GPS, IP, manuel konum + timezone	Platform GPS API + timezone
Bildirim Scheduler	15sn aralikla kontrol, push gonderme	AlarmManager / UNUserNotificationCenter
Service Worker	Push alma, gosterme, tiklama	Firebase Cloud Messaging + yerel bildirim
Veri Modeli	Abonelik, ayarlar, alarm yapisi	SQLite / Hive / SharedPreferences

1.2 Mobil Uygulama Avantajlari

Mobil uygulamada web versiyonundaki bazi kisitlamalar ortadan kalkar. Birincil avantaj, bildirimlerin sunucu tarafinda bir scheduler ile degil, dogrudan cihazin yerel bildirim sistemiyle planlanmasidir. Bu, sunucu bagimlilikini tamamen ortadan kaldirir, zaman kaymasi sorununu cozer ve internet baglantisi olmasa bile bildirimlerin zamaninda gelmesini saglar. Ikincil avantaj, namaz vakti hesaplamanin yerel olarak yapılmasıdır; herhangi bir API cagrisi gerektirmez ve milisaniye hassasiyetinde çalışır.

Ozellik	Web Uygulama	Mobil Uygulama
Bildirim zamanlama	Sunucu scheduler (15sn kontrol)	Yerel OS scheduler (milisaniye hassasiyet)
Internet gereksinimi	Push için zorunlu	Sadece konum/IP için gerekli
Zaman kaymasi riski	Var (sunucu-client gecikme)	Yok (yerel saat bazli)
Hesaplama yeri	Sunucu + istemci	Tamamen yerel
Arka planda calisma	Sinirli (Service Worker)	Tam (AlarmManager / BGTaskScheduler)

2. Veri Modelleri

Uygulamanın tüm kalıcı verisi iki ana kategoride tutulur: kullanıcının konum ve hesaplama ayarları, ve bildirim alarm ayarları. Web versiyonunda bu veriler hem localStorage'da (istemci) hem de PushSubscription tablosunda (sunucu) saklanır. Mobil uygulamada yerel bir veritabanında (SQLite, Hive veya benzeri) tutulması yeterlidir.

2.1 PushSubscription Modeli

Sunucu tarafında her bir cihazın abonelik kaydını tutan model. Mobil uygulamada bu modelin yerini yerel ayarlar ve bildirim planlaması alır, ancak aynı veri yapısı kullanılabilir.

```
model PushSubscription {
  id          String @default(cuid())
  endpoint    String @unique          // Push abonelik endpoint'i
  p256dh     String                  // Encrypt anahtarı (public)
  auth       String                  // Encrypt anahtarı (auth)
  latitude   Float                   // Enlem (derece)
  longitude  Float                   // Boylam (derece)
  timezone   Float                   // Saat dilimi offset (saat, orn: 3 = UTC+3)
  city       String @default("") // Şehir adı
  method     String @default("suleymaniye") // Hesaplama yöntemi
  asrMadhab  String @default("standard") // İkinci mezhebi
  alarms     String @default("{}") // JSON: alarm ayarları
  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt
}
```

2.2 Alarm Ayarları JSON Yapısı

Her namaz vakti için ayrı alarm ayrı tutulur. Bu JSON yapısı, kullanıcının hangi vakitler için bildirim istediğini ve her vakit için kaç dakika önce pre-alarm istediğini tanımlar.

```
interface PrayerAlarmSetting {
  vakit: string;          // Vakit anahtarı: "imsak", "ogle" vb.
  alarm: boolean;         // Ana bildirim açık mı?
  preAlarm: {
    enabled: boolean;     // Pre-alarm açık mı?
    minutes: number;      // Kaç dakika önce bildir?
  };
}

// Örnek kullanım:
{
  "seher": { vakit: "seher", alarm: true, preAlarm: { enabled: true, minutes: 30 } },
  "imsak": { vakit: "imsak", alarm: true, preAlarm: { enabled: true, minutes: 15 } },
  "ogle": { vakit: "ogle", alarm: true, preAlarm: { enabled: false, minutes: 0 } },
  "ikindi": { vakit: "ikindi", alarm: true, preAlarm: { enabled: false, minutes: 0 } },
  "aksam": { vakit: "aksam", alarm: true, preAlarm: { enabled: false, minutes: 0 } },
  "yatsi": { vakit: "yatsi", alarm: true, preAlarm: { enabled: false, minutes: 0 } }
}
```

2.3 Hesaplama Konfigurasyonu

```
interface CalculatorConfig {
  asrType: 'evvel' | 'sani'; // Asr-i Evvel veya Asr-i Sani
  method: CalculationMethod; // Hesaplama yöntemi
  seherAngle: number;        // Seher acisi (varsayılan: -18)
  imsakAngle: number;        // Imsak acisi (SV: -9, Diyanet: -18)
  yatsiAngle: number;        // Yatsi acisi (SV: -9, Diyanet: -17)
  gunesRefraction: number;   // Güneş kırma (varsayılan: 0.833)
```

```
    temkin: number;                // Temkin dakika (SV: 0, Diyanet: 2)
}
```

2.4 Konum Veri Yapisi

```
interface ResolvedLocation {
    latitude: number;    // Enlem (derece)
    longitude: number;   // Boylam (derece)
    city: string;        // Sehir adi
    country: string;     // Ulke adi
    timezone: number;    // UTC offset (saat, orn: 3 = UTC+3)
    timezoneName: string; // IANA timezone adi ("Europe/Istanbul")
}
```

3. Namaz Vakti Hesaplama - Suleymaniye Vakfi Mizan

Suleymaniye Vakfi Mizan metodolojisi, Islam fikhini ve astronomik hesaplamalari birlestiren ozel bir namaz vakti hesaplama sistemidir. Standart astronomik algoritmalar (USNO) temel alinir, ancak imsak ve yatsi acilari diger tum yontemlerden farklidir. Imsak icin -9 derece (Fecr-i Sadik), yatsi icin -9 derece kullanilir. Ayrica temkin sifir olarak uygulanir cunku hesaplamanin kendisi yukari yuvarlama ile temkin etkisini kapsar.

3.1 Adim 1: Julyen Gun Hesaplama

Hesaplamanin ilk adimi, verilen tarihin Julyen Gun Sayisini (JDN) bulmaktir. Julyen Gun Sayisi, M.O. 4713 Ocak 1'den itibaren gecen gun sayisini ifade eder ve astronomik hesaplamalarin temelini olusturur. Onemli nokta: hesaplama kullanicinin yerel tarihine gore yapilir, sunucu saat diliminden bagimsizdir. Bunun icin UTC zaman + timezone offset ile kullanicinin yerel tarihi bulunur.

$$JD = \text{floor}(365.25 \times (Y+4716)) + \text{floor}(30.6001 \times (M+1)) + D + B - 1524.5$$

Burada Y=yil, M=ay, D=gun+0.5 (ogle vakti), B = 2-A+floor(A/4), A=floor(Y/100)

```
// Kullanicinin yerel tarihini hespla
const utcMs = date.getTime();
const userLocalMs = utcMs + timezone * 3600000;
const userLocalDate = new Date(userLocalMs);

// Julyen Gun: yerel tarih + ogle vakti (0.5 gun)
const jd = julianDay(
  userLocalDate.getUTCFullYear(),
  userLocalDate.getUTCMonth() + 1,
  userLocalDate.getUTCDate() + 0.5
);

// J2000.0'dan itibaren yuzyil
const T = (jd - 2451545.0) / 36525.0;
```

3.2 Adim 2: Gunes Astronomik Parametreleri

Julyen yuzyil (T) kullanilarak gunesin astronomik parametreleri hesaplanir. Bu parametreler, gunesin gokyuzundeki konumunu belirlemek icin kullanilir ve tum vakit hesaplamalarinin temelini olusturur.

Parametre	Formul	Aciklama
L0 (Ort. Boylam)	$280.46645 + 36000.76983T + 0.00030327T^2$	Gunesin ortalama ekliptik boylami
M (Ort. Anomali)	$357.5291 + 35999.0503T - 0.0001559T^2$	Gunesin ortalama anomalisi
e (Eksantriklik)	$0.016708617 - 0.000042037T$	Dunyanin yorunge eksantrikligi
C (Denk. Merkezi)	$1.914602\sin(M) + 0.019993\sin(2M) + 0.000269\sin(3M)$	Gunesin ortalama ekliptik boylam merkezi
Lambda (Gercek Boylam)	$L0 + C$	Gunesin gercek ekliptik boylami
Lambda' (Gorunen)	$Lambda - 0.00569 - 0.00478\sin(\omega)$	Nutasyon duzeltmeli
epsilon (Egiklik)	$23.439 - 0.00000036 \times d$ (duzeltilmis)	Ekliptik egiklik acisi
delta (Dik Aciklik)	$\arcsin(\sin(\epsilon) \times \sin(Lambda'))$	Gunes dik acikligi (δ)
EqT (Zaman Denklemi)	$(L0 - RA) / 15$ saat	Saat denklemi

3.3 Adim 3: Ogle (Zeval/Dhuhr) Hesaplama

Ogle vakti, tum vakit hesaplamalarinin referans noktasidir. Gunesin meridyen gecis zamanidir. Zaman denklemi (EqT) ve boylam duzeltmesi ile hesaplanir.

$$Dhuhr = 12 + TimeZone - L/15 - EqT$$

Burada L boylam (derece), TimeZone UTC offset (saat), EqT zaman denklemi (saat). L/15 ifadesi, boylamin saat cinsinden duzeltmesini verir: her 15 derece = 1 saat fark.

3.4 Adim 4: Saat Acisi (Hour Angle) Hesaplama

Saat acisi, gunesin belirli bir yukseklik acisina kac saat uzak oldugunu belirler. Bu fonksiyon imsak, seher, gunes dogusu, akşam ve yatsi vakitlerinin hepsi icin kullanilir. Acı parametresi negatif oldugunda ufuk altini (depression angle) ifade eder.

$$T(\alpha) = (1/15) \times \arccos[(\sin(\alpha) - \sin(\phi) \times \sin(\delta)) / (\cos(\phi) \times \cos(\delta))]$$

Burada alpha=gunes yukseklik acisi, phi=enlem, delta=gunes dik acikligi. $\cos H > 1$ ise gunes hic batmiyor (beyaz geceler), $\cos H < -1$ ise hic dogmuyor.

3.5 Adim 5: Vakit Hesaplama

Her vaket, ogle vakti (dhuhr) referans alinarak saat acisi eklenerek veya cikarilarak hesaplanir. Suleymaniye Vakfi icin kullanilan acilar ve hesaplama yontemi asagidadir:

Vakit	Hesaplama	Acı / Formül
Seher	Dhuhr - T(-18)	-18 derece (Fecr-i Kazib)
Imsak	Dhuhr - T(-9)	-9 derece (Fecr-i Sadik)
Gunes	Dhuhr - T(-0.833)	-0.833 derece (kirma duzeltmeli)
Ogle	Dhuhr	Meridyen gecisi
Ikinci	Dhuhr + A(asr)	Golge formulu (asrHourAngle)
Aksam	Dhuhr + T(-0.833)	Gunes batisi (gunes ile simetrik)
Yatsi	Dhuhr + T(-9)	-9 derece
Yatsi Sonu	Dhuhr + T(-18)	-18 derece

3.6 Adim 6: Asr Golge Formulu

Ikinci vakti, gunesin golge acisina gore hesaplanir. Asr-i Evvel'de golge carpani 1 (nesne golgesi + 1 kat golge), Asr-i Sani'de 2 (nesne golgesi + 2 kat golge) olarak kullanilir. Bu, Hanefi ve Safii mezhepleri arasindaki temel farkliliktir.

$$A(t) = (1/15) \times \arccos[(\sin(\arctan(1/(t+\tan(|\phi-\delta|)))) - \sin(\phi) \times \sin(\delta)) / (\cos(\phi) \times \cos(\delta))]$$

Burada t=golge carpani (1=evvel, 2=sani), phi=enlem, delta=gunes dik acikligi.

3.7 Adim 7: Mizan Kurali (Yuksek Enlem)

Yuksek enlem bolgelerinde (kutuplara yakin), gunes belirli acilara inmedigi icin bazen vakitler hesaplanamaz. Bu durumda Suleymaniye Vakfi'nin Mizan kurali uygulanir. Mizan, geceyi fikhi

acidan uce boler: $P1$ (Aksam-Yatsi sonu) = gece/4, $P2$ (Yatsi sonu-Imsak) = gece/2 (uyku/teheccud), $P3$ (Imsak-Gunes) = gece/4. Kosul: $P1 = P3$ ve $P2 > P1$ olmalidir. Bu bolme, gece suresi (gunes dogusu - aksam) baz alinarak yapilir.

```
function applyMizanRule(aksamHour, gunesHour, imsakHour, yatsiHour):
  nightDuration = gunesHour - aksamHour // ertesi gunse: (24 - aksam) + gunes
  p1 = nightDuration / 4 // Aksam -> Yatsi sonu
  p2 = nightDuration / 2 // Yatsi sonu -> Imsak (uzun uyku)
  p3 = nightDuration / 4 // Imsak -> Gunes
  newYatsiSonu = aksamHour + p1
  newImsak = gunesHour - p3
  newYatsi = aksamHour + nightDuration / 6
```

3.8 Adim 8: Saat Cinsinden Date Nesnesine Donusum

Hesaplanan saat cinsinden degerler (ornegin 12.117 = 12:07) Date nesnelere donusturulur. Bu donusumde onemli nokta, kullanicinin saat dilimi offset'inin dogru uygulanmasidir. Yerel saat UTC'ye cevrilirken timezone cikarilir (yerel 12:07 UTC+3 = UTC 09:07). Ayrica yuvarlama kurallari vardir: bitis vakitleri (gunes, yatsi sonu) asagi yuvarlanir, diger vakitler yukari yuvarlanir (temkin amaclli).

```
function hoursToDate(baseDate, hours, timezone):
  // Kullanicinin yerel tarihini hesapla
  userLocalDate = new Date(baseDate.getTime() + timezone * 3600000)
  year = userLocalDate.getUTCFullYear()
  month = userLocalDate.getUTCMonth()
  day = userLocalDate.getUTCDate()

  for each (key, hour) in hours:
    isEndTime = (key == 'gunes' || key == 'yatsiSonu')
    totalMinutes = hour * 60
    roundedMinutes = isEndTime ? floor(totalMinutes) : ceil(totalMinutes)
    h = floor(roundedMinutes / 60)
    m = roundedMinutes % 60
    // Yerel saat - timezone = UTC timestamp
    result[key] = new Date(Date.UTC(year, month, day, h - timezone, m, 0))
```

4. Diger Hesaplama Yontemleri (Adhan)

Suleymaniye Vakfi disindaki 11 yontem, Adhan kutuphanesi kullanilarak hesaplanir. Adhan, JavaScript tabanlı bir namaz vakti hesaplama kutuphanesidir ve dunyaca kabul goren tum hesaplama yontemlerini icerir. Mobil uygulamada bu kutuphanenin Dart (flutter_prayer_times) veya Swift/Kotlin eslenikleri kullanilabilir. Her yontem, kendine ozgu imsak ve yatsi acilari ile tanimlanir.

4.1 Yontem Parametreleri Tablosu

Yontem	Adhan Metodu	Imsak Acisi	Yatsi Acisi	Hanafi	Temkin
Suleymaniye (Mizan)	Ozel hesaplama	-9	-9	Hayir	0
Diyanet (Türkiye)	Turkey	-18	-17	Hayir	2
MWL	MuslimWorldLeague	-18	-17	Hayir	0
ISNA	NorthAmerica	-15	-15	Hayir	0
Misir	Egyptian	-19.5	-17.5	Hayir	0
Karachi	Karachi	-18	-18	Hayir	0
Ummu'l-Kura	UmmAlQura	-18.5	90dk(saat)	Hayir	0
Dubai	Dubai	-18.2	-18.2	Evet	0
Katar	Qatar	-18	90dk(saat)	Hayir	0
Kuveyt	Kuwait	-18	-17.5	Hayir	0
Singapur	Singapore	-20	-18	Hayir	0
Tahran	Tehran	-17.7	-14(saat)	Hayir	0

4.2 Adhan Hesaplama Akisi

Adhan kutuphanesi ile hesaplama yapilirken izlenen adimlar sunlardir:

1. Coordinates(latitude, longitude) olustur
2. getAdhanParams(method) ile yontem parametrelerini al
3. Ikinci mezhebi ayarla: Hanafi veya Shafii
4. Yuksek enlem kurali: HighLatitudeRule.recommended(coords)
5. Kutup dairesi cozumleme: PolarCircleResolution.AqrabYaum
6. Yuvarlama: Rounding.Up (temkin amaclı)
7. Kullanıcının yerel tarihini hesapla: UTC + timezone offset
8. AdhanPrayerTimes(coords, date, params) olustur
9. UTC sonuclari kullanıcı saat dilimine cevir (toLocalTime)

4.3 Timezone Donusum Fonksiyonu

Adhan kutuphanesi UTC Date nesneleri dondurur. Bu degerlerin kullanıcının yerel saatine cevrilmesi gerekir. Donusum fonksiyonu saat dilimi offset'ini (saat cinsinden) kullanir.

```
function toLocalTime(utcDate, timezoneOffset):
    utcHours = utcDate.getUTCHours()
    utcMinutes = utcDate.getUTCMinutes()
    totalLocalMinutes = (utcHours * 60 + utcMinutes) + (timezoneOffset * 60)
    localHours = floor(totalLocalMinutes / 60) % 24
    localMinutes = totalLocalMinutes % 60
    if localHours < 0: localHours += 24
```



```
// Dogru UTC timestamp: yerel saat - timezone = UTC  
return new Date(Date.UTC(year, month, day, localHours - timezoneOffset, localMinutes, 0))
```

5. Konum Tespiti Algoritması

Uygulama 3 katmanlı bir konum tespiti sistemi kullanır. Öncelik sırasına göre: tarayıcı GPS, IP bazlı konum ve manuel giriş. Mobil uygulamada GPS öncelikli olmalıdır çünkü cihazın yerel konum servisi çok daha hassas ve hızlıdır. IP bazlı konum sadece GPS izni verilmediğinde fallback olarak kullanılır.

5.1 Katman 1: Tarayıcı/GPS Konum

```
function getBrowserLocation():
  if !navigator.geolocation: reject("Desteklenmiyor")
  navigator.geolocation.getCurrentPosition(
    success: (pos) => { lat: pos.coords.latitude, lon: pos.coords.longitude },
    error: reject,
    options: { enableHighAccuracy: false, timeout: 5000, maximumAge: 600000 }
  )
```

Mobil uygulamada: Platform GPS API'leri kullanılır (iOS: CoreLocation, Android: FusedLocationProvider). enableHighAccuracy: false ayarı, hızlı sonuç için Wifi/baz-istasyonu tabanlı konum kullanır. maximumAge: 600000 (10 dakika) önbellek süresi, sık GPS sorgularından kaçınmayı sağlar.

5.2 Katman 2: IP Bazlı Konum (Fallback)

```
async function getIPLocation():
  response = fetch('https://ipapi.co/json/', { timeout: 5000 })
  data = response.json()
  return { lat, lon, city, country }
```

Mobil uygulamada IP bazlı konum servisleri (ipapi.co, ipinfo.io) kullanılabilir. Ancak GPS'e göre çok daha düşük hassasiyetlidir (şehir seviyesi). Varsayılan konum İstanbul'dur.

5.3 Timezone Hesaplama

```
function getTimezoneOffset():
  now = new Date()
  return -now.getTimezoneOffset() / 60 // Saat cinsinden UTC offset

function getTimezoneName():
  return Intl.DateTimeFormat().resolvedOptions().timeZone
```

Mobil uygulamada timezone, platform API'lerinden doğrudan alınır. Dart için: DateTime.now().timeZoneOffset, Swift için: TimeZone.current.secondsFromGMT(). Önemli: namaz vakti hesaplaması timezone offset'inin sayısal değerini kullanır, IANA ismini değil. Ancak DST (yaz saati) değişiklikleri otomatik yansıtılır.

5.4 Reverse Geocoding

Koordinatlardan şehir/ülke bilgisi almak için OpenStreetMap Nominatim API kullanılır. Mobil uygulamada yerel geocoder servisi (iOS CLGeocoder, Android Geocoder) tercih edilmelidir çünkü internet bağlantısı gerektirmez ve daha hızlıdır.

```
async function reverseGeocode(lat, lon):
  response = fetch('https://nominatim.openstreetmap.org/reverse?'
    + 'format=json&lat=' + lat + '&lon=' + lon
    + '&zoom=10&addressdetails=1&accept-language=tr',
    { headers: { 'User-Agent': 'PrayerApp/1.0' }, timeout: 5000 })
  data = response.json()
  city = data.address.city || data.address.town || data.address.village
```

```
country = data.address.country
```

5.5 Konum Onbellek Mekanizmasi

Konum bilgisi localStorage'da 7 gun suresiyle onbelleklenir. Bu, her sayfa yenilemesinde GPS/IP sorgusu yapılmasını onler. Mobil uygulamada SharedPreferences (Android) veya UserDefaults (iOS) kullanılabilir. Onbellek suresi dolunca veya kullanıcı 'yenile' butonuna basarsa tekrar sorgu yapılır.

```
// Onbellek yapisi
interface CachedLocation extends ResolvedLocation {
    _cachedAt: number; // timestamp
}
// Gecerlilik kontrolu: Date.now() - cachedAt > 7 * 24 * 60 * 60 * 1000
```

6. Bildirim Sistemi - Sunucu Tarafi Scheduler

Web uygulamasinda bildirimler sunucu tarafinda bir scheduler ile gonderilir. Bu scheduler, Next.js instrumentation hook'u ile baslatilir ve her 15 saniyede bir calisir. Her calismada veritabanindaki tum abonelikleri tarar, her abone icin namaz vakitlerini hesaplar ve vakti gelenlere push bildirim gonderir. Mobil uygulamada bu sistemin yerini cihazin yerel bildirim planlayicisi alir; sunucu scheduler'a gerek yoktur.

6.1 Scheduler Ana Dongu

```
CHECK_INTERVAL = 15 saniye  
NOTIFICATION_WINDOW = 90 saniye
```

scheduler baslangic:

1. VAPID key'leri kontrol et
2. web-push kutuphanesini yapilandir
3. sentNotifications Map olustur (tekrar engelleme)
4. 10 saniye bekle (sunucu hazirlik)
5. checkAndNotify() calistir
6. setInterval(checkAndNotify, CHECK_INTERVAL) baslat

6.2 checkAndNotify Algoritmasi

```
function checkAndNotify():  
  if isRunning: return // Eszamanli calisma onle  
  isRunning = true  
  
  subscriptions = db.pushSubscription.findMany()  
  now = new Date()  
  
  for each sub in subscriptions:  
    // 1. Hesaplama yontemini ve config'i al  
    method = sub.method  
    config = METHOD_CONFIGS[method]  
  
    // 2. Namaz vakitlerini hesapla  
    calculator = new SuleymaniyePrayerCalculator(config, sub.asrMadhab)  
    result = calculator.calculate(now, { latitude, longitude, timezone, city })  
  
    // 3. Alarm ayarlarini parse et  
    alarms = JSON.parse(sub.alarms)  
    prayerOrder = getPrayerOrder(method)  
  
    // 4. Yerel tarih hesapla (timezone ile)  
    localDateStr = getLocalDateStr(now, sub.timezone)  
  
    // 5. Her vakit icin kontrol  
    for each p in prayerOrder:  
      if !alarms[p.key].alarm: continue  
      prayerTime = result.times[p.key]  
      diff = prayerTime.getTime() - now.getTime()  
  
      // Ana bildirim: vaktin 0-90sn icinde  
      if diff <= 0 AND diff > -90000:  
        notifKey = "prayer-" + p.key + "-" + localDateStr  
        if sentNotifications.has(notifKey): continue  
        sendPush(sub, notifKey, title, body)  
        sentNotifications.set(notifKey, Date.now())  
  
      // Pre-alarm: vaktin X dk oncesi  
      if alarms[p.key].preAlarm.enabled:
```

```

preAlarmTime = prayerTime - preAlarmMinutes * 60000
preAlarmDiff = preAlarmTime - now.getTime()
if preAlarmDiff <= 0 AND preAlarmDiff > -90000:
    notifKey = "prealarm-" + p.key + "-" + localDateStr
    if sentNotifications.has(notifKey): continue
    sendPush(sub, notifKey, title, body)
    sentNotifications.set(notifKey, Date.now())

```

```
isRunning = false
```

6.3 Tekrar Bildirim Engelleme

Aynı namaz vakti için aynı günde birden fazla bildirim göndermeyi önlemek için bir `sentNotifications` Map kullanılır. Anahtar formatı: "prayer-imsak-2026-06-12" veya "prealarm-imsak-2026-06-12". Bu Map 24 saatte bir temizlenir. Mobil uygulamada bu mekanizmaya gerek yoktur çünkü yerel bildirim scheduler her bildirimi sadece bir kez gönderir.

6.4 Timezone Bazlı Tarih Hesaplama

Bildirim tag'lerinde kullanılan tarih, sunucunun UTC tarihi değil, kullanıcının yerel tarihidir. Bu, farklı timezone'lardaki kullanıcıların aynı günde farklı tarihlerle etiketlenmesini sağlar ve bildirim tekrar engelleme mekanizmasının doğru çalışmasını garanti eder.

```

function getLocalDateStr(utcDate, timezone):
    userLocalMs = utcDate.getTime() + timezone * 3600000
    userLocalDate = new Date(userLocalMs)
    return YYYY-MM-DD formatında string

```

6.5 VAPID Key Yönetimi

Push bildirimleri için VAPID (Voluntary Application Server Identification) protokolü kullanılır. Üç anahtar gereklidir: public key (istemciye gönderilir), private key (sunucuda saklanır) ve subject (mailto: veya https: formatında). Mobil uygulamada VAPID gerekli değildir; FCM (Firebase Cloud Messaging) token'ları veya APNs (Apple Push Notification Service) sertifikaları kullanılır.

```

VAPID_PUBLIC_KEY = "BE8XWH3AVzdJv9vTXlKn6le0B6B-fW_fnyDXmJhCwlcZj5..."
VAPID_PRIVATE_KEY = "1oxMwUbsJElntXX056UH8JHv-RzmFatm1XQLT6Cn7dJE"
VAPID_SUBJECT = "mailto:prayer@suleymaniye.com"

```

```
webpush.setVapidDetails(VAPID_SUBJECT, VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY)
```

7. Bildirim Sistemi - Istemci Tarafi

Web uygulamasinda istemci tarafinda iki bildirim mekanizmasi vardir: push bildirim (sunucudan gelen) ve lokal bildirim (istemci tarafinda hesaplanan). Push aboneligi aktifse lokal bildirim devre disi bırakılır çünkü aksi takdirde çift bildirim oluşur. Mobil uygulamada sadece yerel bildirim kullanılır.

7.1 Push Bildirim Alma (Service Worker)

```
self.addEventListener('push', (event) => {
  data = event.data.json()
  title = data.title || 'Namaz Vakti'
  options = {
    body: data.body,
    icon: data.icon || '/favicon.ico',
    tag: data.tag, // Ayni tag'li bildirimleri degistir
    requireInteraction: true, // Kullanici kapatana kadar kal
    vibrate: [200, 100, 200], // Titresim deseni
    data: { url: data.url, prayerKey: data.prayerKey }
  }
  event.waitUntil(self.registration.showNotification(title, options))
})
```

Mobil karsiligi: FCM (Android) ve APNs (iOS) push bildirim alma. requireInteraction karsiligi: bildirim otomatik kapanmaz, kullaci kaydirana kadar kalir. Tag sistemi: ayni tag'li bildirimler birbirini degistirir, bu sayede ayni vaktin birden fazla bildirimi birikmez.

7.2 Bildirime Tiklama Isleme

```
self.addEventListener('notificationclick', (event) => {
  event.notification.close()
  urlToOpen = event.notification.data?.url || '/'

  // Acik pencere varsa odaklan, yoksa yeni pencere ac
  clients.matchAll({ type: 'window' }).then(clientList => {
    for client in clientList:
      if client.url.includes(origin): return client.focus()
    return clients.openWindow(urlToOpen)
  })
})
```

Mobil karsiligi: bildirime tiklandiginda uygulama acilir, ilgili vakit sayfasina yonlendirilir. iOS'ta UNUserNotificationAction, Android'de PendingIntent kullanilir.

7.3 Lokal Bildirim Fallback

Push aboneligi yoksa, istemci tarafinda 1 saniyelik aralikla lokal bildirim kontrolu yapilir. Bildirim penceresi 90 saniyedir (vaktin gecmesinden sonraki 90 saniye icinde). Bu, web ortaminda sunucu baglantisi olmadan da bildirim gosterilmesini saglar.

```
// Lokal bildirim kontrol dongusu (1 saniye aralik)
setInterval(() => {
  now = new Date()
  for each p in prayerOrder:
    prayerTime = prayerTimes[p.key]
    diff = prayerTime.getTime() - now.getTime()

    // Ana bildirim: 0 ile -90sn arasi
    if diff <= 0 AND diff > -90000 AND !notifiedRef.has(key):
      notifiedRef.add(key)
```

```
        navigator.serviceWorker.ready.then(reg =>
            reg.showNotification(title, { tag, requireInteraction: true, vibrate: [200,100,200] })
        )

        // Pre-alarm: vaktin X dk oncesi
        preAlarmTime = prayerTime - preAlarmMinutes * 60000
        preAlarmDiff = preAlarmTime - now.getTime()
        if preAlarmDiff <= 0 AND preAlarmDiff > -90000 AND !notifiedRef.has(preKey):
            // Pre-alarm goster
    }, 1000)
```

8. Hicri Tarih Donusumu Algoritmasi

Miladi tarihten Hicri tarihe donusum icin Kuwaiti algoritmasi kullanilir. Bu algoritma, Julyen Gun Sayisi uzerinden hesaplama yapar ve yaklasik bir sonuc verir (astronomik hesaba dayali degildir). Ramazan vb. ozel aylarin baslangicinda 1-2 gun sapma olabilir; kesin tarih Diyanet ilmi kurulu tarafindan hilal gozlemi ile belirlenir.

```
function gregorianToHijri(date):
  gd = date.day, gm = date.month, gy = date.year
  if gm < 3: gy -= 1; gm += 12

  a = floor(gy / 100)
  b = 2 - a + floor(a / 4)
  jd = floor(365.25 * (gy + 4716)) + floor(30.6001 * (gm + 1))
    + gd + b - 1524.5

  // Hicri hesaplama
  l = floor(jd - 1948439.5 + 10632)
  n = floor((l - 1) / 10631)
  l2 = l - 10631 * n + 354
  j = floor((10985 - l2) / 5316) * floor((50 * l2) / 17719)
    + floor(l2 / 5670) * floor((43 * l2) / 15238)
  l3 = l2 - floor((30 - j) / 15) * floor((17719 * j) / 50)
    - floor(j / 16) * floor((15238 * j) / 43) + 29
  mH = floor((24 * l3) / 709) // Hicri ay
  dH = l3 - floor((709 * mH) / 24) // Hicri gun
  yH = 30 * n + j - 30 // Hicri yil

  return { year: yH, month: mH, day: dH,
    monthName: HIJRI_MONTHS[mH - 1] }
```

8.1 Hicri Ay Isimleri

No	Ay Adi	No	Ay Adi
1	Muharrem	7	Recep
2	Safer	8	Sabam
3	Rebiulevvel	9	Ramazan
4	Rebiulâhir	10	Sevval
5	Rumusâni (Cemaziyelevvel)	11	Zilkade
6	Cemaziyelâhir	12	Zilhicce

9. Aktif Vakit ve Geri Sayim Algoritmasi

Kullanicinin o an hangi vakitte oldugunu ve bir sonraki vakte kalan sureyi hesaplamak icin getActivePrayer fonksiyonu kullanilir. Bu fonksiyon, namaz vakitlerini sondan basa dogru tarayarak guncel zamandan onceki en son vakti bulur. Gece yarisi gecis durumu ozel olarak ele alinir.

```
function getActivePrayer(times, now, method):
  order = getPrayerOrder(method) // SV: 8 vakit, diger: 6 vakit
  keys = order.map(p => p.key)

  // Sondan basa tara: guncel zamandan onceki son vakti bul
  for i = keys.length - 1 downto 0:
    if now >= times[keys[i]]:
```



```

    active = order[i]
    nextIndex = (i + 1) % order.length
    next = order[nextIndex]

    nextTime = times[keys[nextIndex]]
    // Gece yarisi gecis: ilk vaktin ertesi gun
    if nextIndex == 0:
        nextTime = nextTime + 24 hours

    timeUntilNext = nextTime.getTime() - now.getTime()
    return { active, next, timeUntilNext }

// Hicbir vakit bulunamazsa (ilk vakitten once)
return { active: order[son], next: order[0], timeUntilNext }

```

9.1 Vakit Sıralamasi

Suleymaniye Vakfi (8 Vakit)	Diger Yontemler (6 Vakit)
1. Seher Vakti	1. Imsak
2. Sabah Namazi (Imsak)	2. Gunes
3. Sabah Namazi Sonu (Gunes)	3. Ogle
4. Ogle	4. Ikindi
5. Ikindi	5. Aksam
6. Aksam	6. Yatsi
7. Yatsi	
8. Yatsi Sonu	

10. Mobil Uygulama Teknik Semasi

Bu bolum, mevcut web algoritmalarının mobil uygulamaya nasıl aktarılacağını detaylandırır. Flutter (Dart) öncelikli olarak önerilir çünkü hem iOS hem Android'i tek kod tabaniyla destekler ve Adhan kutuphanesinin Dart portu mevcuttur. Ancak React Native veya native (Swift/Kotlin) seçenekleri de değerlendirilmiştir.

10.1 Platform Karsilastirmasi

Ozellik	Flutter (Dart)	React Native	Native (Swift/Kotlin)
Hesaplama kutuphanesi	adhan_dart (port)	adhan-js (same)	Ozel implementasyon
Yerel bildirim	flutter_local_notifications	react-native-push-notification	AlarmManager / UNUserNotificationCenter
GPS konum	geolocator	react-native-geolocation	CoreLocation / FusedLocation
Timezone	timezone package	Intl.DateTimeFormat	TimeZone.current / ZoneId
Veritabani	Hive / SQLite	AsyncStorage / SQLite	CoreData / Room
Offline calisma	Tam destek	Tam destek	Tam destek
Gelistirme suresi	Orta	Orta	Uzun (2x kod)

10.2 Yerel Bildirim Planlama Stratejisi

Mobil uygulamada en kritik fark, bildirimlerin sunucu scheduler yerine cihazın yerel bildirim sistemiyle planlanmasıdır. Bu yaklaşım internet bağlantısı gerektirmez, zaman kayması olmaz ve pil tüketimini minimumda tutar. Her gün gece yarısı (veya uygulama açıldığında) ertesi günün tüm namaz vakitleri hesaplanır ve her vaktin tam zamanına yerel bildirim schedule edilir.

10.2.1 Android: AlarmManager + WorkManager

```
// Her gün gece yarısı çalışan WorkManager
class DailyPrayerWorker(appContext, params):
    override fun doWork():
        // 1. Vakitleri hesaplama
        times = PrayerCalculator.calculate(today, location, config)
        // 2. Her vaktin AlarmManager set et
        for (prayerKey, prayerTime) in times:
            if alarms[prayerKey].alarm:
                scheduleExactAlarm(prayerTime, prayerKey)
            if alarms[prayerKey].preAlarm.enabled:
                preTime = prayerTime - alarms[prayerKey].preAlarm.minutes * 60000
                scheduleExactAlarm(preTime, "prealarm_${prayerKey}")
        // 3. Ertesi gün için tekrar planla
        scheduleNextDayWork()
        return Result.success()
```

10.2.2 iOS: UNUserNotificationCenter

```
// Uygulama açıldığında veya gece yarısı çalışır
func schedulePrayerNotifications(times: PrayerTimes, alarms: [String: AlarmSetting]) {
    UNUserNotificationCenter.current().removeAllPendingNotificationRequests()

    for (key, time) in times {
        if alarms[key]?.alarm == true {
            let content = UNMutableNotificationContent()
            content.title = "\(key.label) Vakti"
            content.body = "\(key.label) vakti geldi: \(formatTime(time))"
            content.sound = .default
        }
    }
}
```

```

        content.categoryIdentifier = "PRAYER_\(key)"

        let trigger = UNCalendarNotificationTrigger(
            dateMatching: Calendar.current.dateComponents([.year,.month,.day,.hour,.minute], from: time)
            repeats: false
        )
        let request = UNNotificationRequest(identifier: "prayer_\(key)_\(todayStr)", content: content, trigger: trigger)
        UNUserNotificationCenter.current().add(request)
    }
}
}

```

10.3 Offline Hesaplama Akisi

Namaz vakti hesaplama tamamen offline calisir. Herhangi bir API cagrisi gerektirmez. Tek gereksinim: konum koordinatlari (enlem, boylam) ve timezone offset. Bu degerler bir kez alindikten sonra yerel olarak saklanir ve internet baglantisi olmasa bile her gun yeni vakitler hesaplanabilir. Sadece sehir adi guncelleme (reverse geocoding) ve IP bazli konum icin internet gerekir.

```

// Mobil offline hesaplama akisi
1. Uygulama acilir
2. Kayitli konumu oku (Hive/SharedPreferences)
3. Kayitli konum yoksa -> GPS/IP ile konum al
4. PrayerCalculator.calculate(now, location, config)
5. Vakitleri goster + bildirimleri planla
6. Timer ile her saniye geri sayim guncelle
7. Gece yarisi geldiginde -> yeni gunun vakitlerini hesapla

```

10.4 Performans Notlari

Namaz vakti hesaplama, 12 astronomik fonksiyonun sirayla calistirilmasini icerir. Hesaplama maliyeti cok dusuktur: tek bir hesaplama 1ms'den kısa surer. Bu nedenle her saniye geri sayim guncellenirken yeniden hesaplama yapmak yerine, hesaplanan vakitler onbellege alinir ve sadece tarih degisikliginde (gece yarisi) yeniden hesaplanir. Bildirim planlamasi ise gunde bir kez yapilir ve cihazin isletim sistemi zamaninda tetikler.

Islem	Frekans	Maliyet	Not
Vakit hesaplama	Gunde 1 kez (gece yarisi)	< 1ms	Sonuclar onbellege alinir
Geri sayim guncelleme	Her saniye	< 0.01ms	Onbellekli vakitlerle hesap
Konum tespiti	Uygulama acilista	100ms-2sn	GPS bagimli
Bildirim planlama	Gunde 1 kez	< 10ms	OS scheduler'a kayit
Reverse geocoding	Konum degisince	200-500ms	Istege bagli, offline calismaz

ONEMLI NOT: Bu dokumandaki tum algoritmalar, mevcut web uygulamasinin birebir implementasyonudur. Mobil uygulamaya aktarilirken hesaplama mantiginda hicbir degisiklik yapilmamalidir; sadece bildirim ve konum mekanizmalari platforma uygun sekilde uyarlanmalidir.